PROJECT 31A — FRONTEND APPLICATION BIBLE (DOCUMENT 6)

Version 1.0.0

Classification: CONFIDENTIAL — FOR INTERNAL DEVELOPMENT AND DUE DILIGENCE ONLY

---

1. TECHNOLOGY STACK OVERVIEW

The frontend of Project 31A is built as a single-page, highly interactive dashboard utilizing **Next.js 15+ (App Router)** and **React 19**. It is styled using **TailwindCSS v4** and incorporates state-of-the-art UI primitives and motion libraries for a premium, SRE-grade operational experience.

1.1 Core Frontend Dependencies

- **Framework:** Next.js 16.2.6 (App Router)
- **UI Runtime:** React 19.2.4 & React DOM 19.2.4
- **State Management:** Zustand 5.0.13 (for local client state, sidebar toggle, user context)
- **Data Fetching & Cache:** TanStack React Query 5.100.10 (manages all API data caching, polling, and prefetching)
- **Styling:** TailwindCSS v4 with `@tailwindcss/postcss`
- **UI Primitives:** Radix UI (`@radix-ui/react-*` for Dialog, Dropdown, Tabs, Tooltip, Avatar, Select)
- **Icons:** Lucide React 1.14.0
- **Animations:** Framer Motion 12.38.0 (handles micro-interactions, page transitions, and drawer sliders)
- **Data Visualization:** Recharts 3.8.1 (renders operational health, volatility snapshots, and SRE queues)
- **Diagrams & Graphs:** `@xyflow/react` 12.10.2 (formerly React Flow; renders competitor mapping and prospect graph nodes)
- **Form Management:** React Hook Form 7.76.1 with `@hookform/resolvers` and Zod 4.4.3 validation

1.2 Build & Dev Server Configuration

The development server runs via Turbopack and is configured to bind to port `3002` to avoid conflicts with other development utilities.

- **Start Dev Server:** `npm run dev` (executes `next dev -p 3002`)
- **Production Build:** `npm run build`
- **Execution Run:** `npm run start`

---

2. APPLICATION STRUCTURE & ROUTING

The application adheres to the Next.js App Router paradigm under the `frontend/src/app` directory. The routing structure is organized under a main `/dashboard` root route group.

```
frontend/src/
  ├── app/
  │   ├── layout.tsx             # Root layout and global provider wrapper
  │   ├── page.tsx               # Login / Entry Gate page
  │   ├── providers.tsx          # Clerk, React Query, and UI theme providers
  │   └── dashboard/
  │       ├── layout.tsx         # Dashboard frame, sidebar navigation, top bar
  │       ├── page.tsx           # Operational Overview Command Center
  │       ├── campaigns/         # Campaign List and CRUD wizard
  │       ├── prospect-list/     # Prospect database page
  │       ├── prospect-graph/    # Interactive XYFlow prospect relation network
  │       ├── approvals-center/  # Campaign approval queue
  │       ├── communication-hub/ # Live outreach feed and templates review
  │       ├── citation-operations/# Directory listing builder
  │       ├── temporal/          # Temporal worker queue depths SRE monitor
  │       └── ...                # Additional intelligence modules (74 total)
```

---

3. CORE PAGE INVENTORY & FUNCTIONALITY

With 74 subdirectories under `/dashboard`, the interface maps the backend's comprehensive API surface. The primary workspace layouts are catalogued below:

3.1 Command Center Dashboard (`/dashboard`)

- **Purpose:** The operations cockpit. Renders live system state, campaign run progress, recent alerts, and action center priority notifications.
- **Components:**

- Health overview widget (SSE-backed status circles for PostgreSQL, Kafka, Temporal).
- Priority task lists.
- Anomaly forecast graphs (Recharts).

3.2 Campaign Manager (`/dashboard/campaigns`)

- **Purpose:** Lists current campaigns and hosts the multistep launch modal.
- **Features:**
  - Interactive table showing state transitions (draft -> prospecting -> awaiting_approval -> active).
  - Progress bars displaying `acquired_link_count` against `target_link_count`.
  - Trigger buttons to pause, resume, or force-recover execution runs.

3.3 Prospect Explorer & Graph (`/dashboard/prospect-list` & `/dashboard/prospect-graph`)

- **Purpose:** Detail view of discovered domains.
- **Features:**
  - List view: Filter prospects by spam score, domain authority, and contact status.
  - Graph view (`@xyflow/react` implementation): Renders competitor domain links to prospects. Visually isolates link farms and spam nodes as high-density clusters.

3.4 Approval Center (`/dashboard/approvals-center` & `/dashboard/approvals`)

- **Purpose:** Displays blocking approval requests.
- **Features:**
  - Gate 1 View: Displays list of discovered target prospects with checkbox selection. Operator can select all, edit contact details, or reject specific domains before clicking "Approve outreach".
  - Gate 2 View: Tiptap text editor showing generated email templates. Operator can tweak text inline or click "Regenerate" before authorizing dispatch.

3.5 Communication Hub (`/dashboard/communication-hub`)

- **Purpose:** Live view of outreach threads.
- **Features:**
  - Shared inbox view: Displays sent emails, tracking pixel open alerts, and incoming replies.
  - Reply classification markers (interested, not interested, pricing request) generated by LLM analysis.

3.6 SRE Sytem Console (`/dashboard/temporal` & `/dashboard/advanced-sre`)

- **Purpose:** developer diagnostic panel.
- **Features:**

- Renders Temporal task queue worker capacities.
- Provides a kill switch panel enabling administrators to block email dispatch or LLM calls instantly.

---

4. API INTEGRATION & DATA FLOW PATTERN

The application communicates with the backend exclusively via an asynchronous API client configured in `frontend/src/lib/api.ts`.

4.1 Base Client Wrapper

```
import axios from 'axios';

  export const apiClient = axios.create({
    baseURL: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000/api/v1',
    timeout: 10000,
  });

  // Interceptor to inject Clerk Auth bearer token
  apiClient.interceptors.request.use(async (config) => {
    const token = await window.Clerk?.session?.getToken();
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  });
```

4.2 React Query Query/Mutation Pattern

All CRUD actions utilize React Query hooks. Example hooks for campaigns:

```
// Fetching Campaigns
  export const useCampaigns = (tenantId: string) => {
    return useQuery({
      queryKey: ['campaigns', tenantId],
      queryFn: async () => {
        const res = await apiClient.get(`/campaigns?tenant_id=${tenantId}`);
        return res.data.data;
      },
    });
  };

  // Launching a Campaign
  export const useLaunchCampaign = () => {
    const queryClient = useQueryClient();
    return useMutation({
      mutationFn: async (campaignId: string) => {
        const res = await apiClient.post(`/campaigns/${campaignId}/launch`);
        return res.data.data;
      },
      onSuccess: (data, campaignId) => {
        queryClient.invalidateQueries({ queryKey: ['campaigns'] });
      },
    });
  };
```

---

5. STATE MANAGEMENT STRUCTURE

The frontend implements a hybrid state architecture:

1. **Server State (React Query):** Cached API responses, background prefetching, and query invalidation.
2. **Local Component State (React `useState`):** Form validation, local open/closed dialog states, slider positioning.
3. **Global Application State (Zustand):**
   - **Sidebar Store:** Manages menu collapse toggles and nav grouping settings.
   - **Tenant Store:** Holds the current active `tenant_id` and `client_id` selected in the global top bar dropdown.
   - **Notification Store:** Maintains temporary screen notifications ("sonner" alerts) triggered by incoming WebSocket or SSE events.

---

6. CLERK AUTHENTICATION INTEGRATION

User login, registration, and tenant scoping are managed via the **Clerk** SDK.

- **Provider Wrapper:** The layout wraps pages in `<ClerkProvider>`.
- **Protected Paths:** Routes under `/dashboard` check session validity. Unauthenticated requests redirect to the entry gate (`/`).
- **Organization Switcher:** Clerk's organization switcher maps to tenant records. Selecting a new Clerk organization updates the Zustand Tenant Store, invalidates the active React Query cache, and reloads the API request context with the new `tenant_id` query parameter.